SECUR.IA

L'Intelligence Artificielle au Service de la Sécurité Publique

Guide de Conception, d'Architecture et de Déploiement

Transformation Numérique des Administrations et des Forces de l'Ordre en Afrique

LEMAGEKEN & CO
Technologie Souveraine & Innovation

ÉDITION SPÉCIALE - JUIN 2026

Préface

La sécurité est le pilier fondamental sur lequel repose tout développement économique, social et humain. À l'ère de la transformation numérique, les forces de sécurité et les administrations publiques font face à des défis d'une complexité sans précédent : croissance démographique, urbanisation rapide, menaces hybrides et besoin d'une réactivité accrue. Face à ces enjeux, les technologies traditionnelles ne suffisent plus.

C'est de ce constat qu'est né **SECUR.IA**. Ce projet incarne une vision audacieuse : mettre l'intelligence artificielle générative et les technologies mobiles de pointe directement entre les mains des agents de terrain. Conçu initialement pour répondre aux réalités opérationnelles des forces de police africaines, et notamment au Cameroun sous l'égide de l'assistant intelligent **Pocam237**, SECUR.IA n'est pas seulement une application mobile ; c'est un écosystème de souveraineté numérique.

> *"L'innovation technologique n'est pas un luxe pour l'Afrique, c'est le raccourci nécessaire pour garantir la paix, la justice et la sécurité de nos populations."*
>
> **Comité de Direction Technique, SECUR.IA**

Cet ouvrage de référence a été conçu pour être à la fois un manifeste stratégique et un guide technique pas à pas. Que vous soyez un décideur public cherchant à moderniser vos administrations, un architecte logiciel désireux de comprendre l'intégration de modèles d'IA multimodaux en mode déconnecté, ou un développeur souhaitant reproduire et adapter la solution dans son propre pays, ce livre vous offre toutes les clés de compréhension et d'implémentation.

Table des Matières

Chapitre 1 : Genèse et Vision Stratégique du Projet 4

Chapitre 2 : Conception Opérationnelle et Besoins Terrain 7

Chapitre 3 : Architecture Système et Choix Technologiques 10

Chapitre 4 : Guide de Développement de l'Application Flutter 13

Chapitre 5 : Intégration et Sécurisation de l'Écosystème Firebase 18

Chapitre 6 : Modèle Économique, Commercialisation et Impact 23

Chapitre 7 : Feuille de Route et Perspectives d'Évolution 27

Chapitre 1 : Genèse et Vision Stratégique du Projet

1.1 Introduction au contexte de la Sécurité Publique en Afrique

Le continent africain traverse une phase de mutation rapide caractérisée par une croissance urbaine exponentielle et une adoption massive des technologies mobiles. Cependant, les infrastructures de sécurité publique peinent parfois à suivre cette transition numérique. Dans de nombreuses administrations et forces de l'ordre, l'accès à l'information critique (annuaires professionnels, textes juridiques, guides de procédures opérationnelles) reste fragmenté, obsolète ou difficilement accessible sur le terrain.

Les agents de police, de gendarmerie ou de sécurité civile se retrouvent souvent isolés face à des situations juridiques complexes ou des besoins urgents de communication avec d'autres unités. C'est dans ce contexte de fracture informationnelle que la transformation digitale s'impose non pas comme une simple modernisation administrative, mais comme un impératif de souveraineté et d'efficacité opérationnelle.

1.2 Le Concept SECUR.IA : Mettre l'IA au service de la sécurité

SECUR.IA est une plateforme numérique innovante qui utilise l'intelligence artificielle générative et les technologies mobiles pour combler ce fossé. L'objectif fondamental est d'offrir à chaque agent de terrain un assistant intelligent capable de l'orienter instantanément dans ses missions quotidiennes : consultation sécurisée de l'annuaire professionnel, assistance juridique sur le Code Pénal et la Procédure Pénale, et guidage dans les procédures de rédaction des procès-verbaux et rapports administratifs.

> **La Souveraineté de l'Information**

> La sécurité d'un État repose sur le contrôle et la confidentialité de ses données de communication. SECUR.IA conçoit des architectures qui garantissent que l'annuaire professionnel reste hermétique aux intrusions extérieures tout en exploitant les capacités cognitives des modèles de langage de pointe.

1.3 Focus sur le Cameroun : L'assistant intelligent Pocam237

Le prototype opérationnel de la solution a été déployé pour répondre aux spécificités de la Police Nationale camerounaise sous le nom de code **Pocam237**. Cet assistant virtuel intègre les particularités du système administratif et judiciaire du Cameroun.

Pocam237 répond de manière précise et contextualisée sur le droit camerounais (Code de Procédure Pénale camerounais, décrets relatifs à la Sûreté Nationale) et structure ses réponses selon les normes de rédaction administrative en vigueur dans le pays. Il s'agit d'un exemple concret d'adaptation locale d'une technologie globale.

1.4 L'impératif de l'accès déconnecté

Sur le terrain opérationnel, la connectivité internet est un défi majeur (zones blanches, coupures de réseau, saturation). Un assistant de sécurité qui cesserait de fonctionner dès que la connexion internet est coupée serait inutilisable et dangereux.

SECUR.IA intègre dès sa conception une architecture résiliente : l'annuaire téléphonique professionnel et les données de contact essentielles sont stockés localement sur le téléphone dans une base de données optimisée. Si le réseau est perdu, l'application bascule automatiquement en mode hors-ligne, permettant la consultation à 100% de l'annuaire, tandis que le chatbot IA suspend ses requêtes réseau proprement en informant l'agent avec sérénité.

Fonctionnalité	Mode En Ligne (Connecté)	Mode Hors-Ligne (Déconnecté)
Annuaire Professionnel	Disponible (Lecture et recherche en temps réel)	Disponible à 100% (Base SQLite locale synchronisée)

Fonctionnalité	Mode En Ligne (Connecté)	Mode Hors-Ligne (Déconnecté)
Assistant IA Pocam237	Disponible (Gemini 2.5 Multimodal)	Indisponible (Bannière d'alerte et blocage propre)
Envoi de Pièces Jointes	Disponible (Images, Documents, Photos)	Désactivé proprement

Chapitre 2 : Conception Opérationnelle et Besoins Terrain

2.1 Le Cahier des Charges Opérationnel

La conception de SECUR.IA repose sur une méthodologie centrée sur l'utilisateur de terrain. Pour définir les spécifications de l'application, des ateliers ont été menés avec des agents des forces de l'ordre. Trois exigences fondamentales ont émergé de ces échanges :

- **Rapidité d'exécution** : L'agent doit pouvoir trouver le contact d'une unité ou d'un collègue en moins de 3 secondes en situation d'urgence.
- **Simplicité de l'interface** : Pas de menus complexes ou de boutons superflus. L'interface doit être épurée, contrastée et utilisable sous le soleil ou sous la pluie.
- **Sécurité des données** : Les numéros professionnels ne doivent pas être visibles par le grand public ni stockés dans le répertoire personnel du téléphone pour éviter les fuites.

2.2 Scénarios d'usage sur le terrain

Scénario A : Le contrôle routier ou l'interpellation en zone isolée

Lors d'un contrôle de routine, un officier de police judiciaire (OPJ) suspecte une infraction spécifique liée au transport de marchandises sensibles, mais a un doute sur l'article exact de la loi applicable ou la procédure d'interpellation immédiate. En ouvrant l'assistant IA Pocam237, il pose sa question en langage naturel. L'IA lui répond instantanément en lui donnant le **Contexte Légal** (loi applicable au Cameroun), la **Procédure à suivre** étape par étape pour sécuriser l'interpellation, et les **Avertissements** à respecter pour éviter les vices de procédure.

Scénario B : La recherche urgente de renforts ou d'un service spécialisé

Une patrouille fait face à un incident majeur nécessitant la coordination immédiate avec les services de la Direction Régionale de la Police Judiciaire (DRPJ) ou un commissariat d'arrondissement spécifique. Grâce à la structure hiérarchisée par Régions et par Villes de

l'annuaire local, l'agent trouve le contact direct en quelques clics sans connexion internet, et peut lancer l'appel directement depuis l'application.

> *"Dans notre métier, l'incertitude juridique ou l'impossibilité de joindre une unité en renfort peut transformer une simple intervention en drame. Avoir un outil fiable en mode déconnecté change complètement la donne."*
>
> **Inspecteur de Police, Cameroun**

2.3 Structuration de l'interface utilisateur (UI)

Pour répondre aux contraintes du terrain, l'application est découpée en 5 onglets majeurs accessibles depuis une barre de navigation inférieure animée et haut de gamme :

1. **Annuaire** : Navigation visuelle et intuitive à travers les Régions du Cameroun (représentées par des cartes d'arrière-plan distinctives) puis par villes et par unités.
2. **Chat IA (Pocam237)** : Espace de discussion interactif et multimodal avec l'assistant IA, incluant la possibilité de prendre des photos, de joindre des documents administratifs et d'analyser des situations complexes.
3. **Informations** : Espace institutionnel présentant la mission, la vision de SECUR.IA et les contacts du support technique direct via WhatsApp.
4. **Configuration** : Paramètres de l'application permettant aux administrateurs de configurer le mode de synchronisation de la base de données.
5. **Recherche** : Moteur de recherche globale par mot-clé (nom, service, grade, ville) sur l'intégralité de l'annuaire de contacts, fonctionnant de manière instantanée en local.

Chapitre 3 : Architecture Système et Choix Technologiques

3.1 Pourquoi le choix du Framework Flutter ?

Le choix de **Flutter** (développé par Google) comme technologie de développement d'applications mobiles multiplateformes a été guidé par plusieurs critères critiques d'ingénierie :

- **Base de code unique** : Flutter permet de compiler nativement pour Android et iOS à partir d'un seul code source écrit en Dart. C'est un avantage financier et technique majeur pour les administrations publiques.
- **Performances graphiques (Impeller Engine)** : Les interfaces de Flutter sont dessinées directement sur le GPU du téléphone. Cela garantit des animations fluides (à 60 ou 120 FPS) et un rendu pixel-perfect, même sur des téléphones d'entrée de gamme couramment utilisés en Afrique.
- **Écosystème de paquets** : L'écosystème pub.dev propose des bibliothèques robustes et maintenues pour interagir avec le matériel natif (appareil photo pour la prise de pièces jointes, base de données SQLite, capteurs réseau).

3.2 Stockage Local Hybride : SQLite et JSON

Pour relever le défi de l'accès hors-ligne, SECUR.IA utilise une stratégie de stockage hybride :

1. **Base de données SQLite (via sqflite)** : SQLite est un moteur de base de données relationnelle léger, intégré et autonome. Nous l'utilisons pour stocker la mémoire de conversation locale (sessions de chat et messages) de l'utilisateur sur son téléphone, assurant la persistance des discussions avec Pocam237.
2. **Fichier JSON local d'annuaire** : L'annuaire professionnel initial est embarqué directement dans les assets sous forme de fichier JSON structuré (`annuaire.json`). Ce fichier est chargé en mémoire au premier démarrage et inséré dans la base SQLite locale pour permettre des recherches indexées ultra-rapides sans aucun accès réseau.

> **Mémoire Conversationnelle et Évolution de Schéma**
>
> La base de données locale intègre des mécanismes de migration automatiques (`onUpgrade`). Lors du passage de la version 1 à la version 2, les tables de messages ont été modifiées dynamiquement pour intégrer les colonnes de pièces jointes (`attachment_path`, `attachment_type`), garantissant la conservation de l'historique utilisateur sans perte de données.

3.3 L'intégration Multimodale de l'IA (Gemini 2.5 Flash)

Pour propulser l'assistant intelligent Pocam237, SECUR.IA s'appuie sur le modèle de langage **Gemini 2.5 Flash** de Google. Ce modèle a été sélectionné pour ses caractéristiques exceptionnelles :

- **Fenêtre de contexte gigantesque** : Permet d'envoyer l'historique complet de la discussion et de grands documents textes pour analyse.
- **Capacités multimodales natives** : Le modèle analyse directement les images encodées en Base64 sans nécessiter de modèle de vision externe tiers, permettant à l'agent de photographier une pièce justificative ou une situation sur le terrain et d'en discuter.
- **Coût et Rapidité** : Le modèle Flash est ultra-rapide (faible latence) et dispose d'un coût d'exploitation extrêmement bas, idéal pour des budgets d'administrations publiques.

3.4 La brique Firebase : Authentification et Sécurité

Firebase assure la gestion sécurisée du backend de l'application :

- **Firebase Authentication** : Gère l'accès sécurisé des utilisateurs via e-mail/mot de passe. La session est persistée en local (`AuthWrapper`), évitant aux agents de devoir se reconnecter à chaque ouverture de l'application.
- **Cloud Firestore** : Utilisé comme base de données Cloud en temps réel pour le contrôle de version (`app_config/version_control`) et le stockage de la clé d'API Gemini. Grâce à la persistance hors-ligne Firestore, l'application reste fluide même en cas de micro-coupures réseau.

3.5 Diagramme d'Architecture et flux de données

Le flux d'information de l'application suit une structure étanche et asymétrique :

Composant	Technologie	Rôle et Flux de Données
Interface Mobile (UI)	Flutter (Dart)	Affiche l'annuaire, le chat, gère la capture photo et les documents.
Base de Données Locale	SQLite (sqflite)	Sauvegarde locale des fils de discussion et données de contacts.
Contrôle de Version & Clé API	Firebase Firestore	Vérifie la version installée et récupère la clé API Gemini de manière dynamique.
Moteur Cognitif (IA)	API Gemini 2.5 (HTTPS)	Reçoit le prompt système, l'historique, la question et les pièces jointes, puis renvoie la réponse structurée.

Chapitre 4 : Guide de Développement de l'Application Flutter

4.1 Le point d'entrée : main.dart

Le fichier `main.dart` initialise les briques fondamentales de l'application : liaison des widgets de Flutter, initialisation de Firebase, et surtout configuration de la persistance locale de Cloud Firestore pour garantir le fonctionnement hors-ligne.

```
import 'package:flutter/material.dart';
import 'package:provider/provider.dart';
import 'package:firebase_core/firebase_core.dart';
import 'package:cloud_firestore/cloud_firestore.dart';
import 'package:securia/wrappers/auth_wrapper.dart';
import 'firebase_options.dart';
import 'package:securia/app.dart';
import 'package:securia/config/app_config.dart';

void main() async {
  WidgetsFlutterBinding.ensureInitialized();

  // Initialisation de Firebase avec les options de la plateforme courante
  await Firebase.initializeApp(options: DefaultFirebaseOptions.currentPlatform);

  // ACTIVATION DE LA PERSISTANCE HORS-LIGNE FIRESTORE
  FirebaseFirestore.instance.settings = const Settings(
    persistenceEnabled: true,
    cacheSizeBytes: Settings.CACHE_SIZE_UNLIMITED,
  );

  runApp(
    ChangeNotifierProvider(
      create: (context) => AppConfig(),
      child: const MyApp(homePage: AuthWrapper()),
    ),
  );
}
```

4.2 Le contrôle de mise à jour intelligent : version_helper.dart

Ce composant interroge Firestore au démarrage pour vérifier la version de l'application installée. Si une nouvelle version est obligatoire en ligne, un dialogue non fermable (grâce à `PopScope`) s'affiche, bloquant l'accès à l'application et forçant le téléchargement de l'APK.

```
import 'package:cloud_firestore/cloud_firestore.dart';
import 'package:flutter/material.dart';
import 'package:url_launcher/url_launcher.dart';

class VersionHelper {
  // Version installée localement sur l'appareil
  static const String currentVersion = "1.0.0";

  // Stockage temporaire en mémoire de la clé API Gemini
  static String geminiApiKey = "";

  /// Interroge Firestore et affiche le dialogue de mise à jour obligatoire si nécessaire
  static Future<void> checkVersion(BuildContext context) async {
    try {
      final doc = await FirebaseFirestore.instance
          .collection('app_config')
          .doc('version_control')
          .get();

      if (!doc.exists) return;
      final data = doc.data();
      if (data == null) return;

      final String latestVersion = data['current_version'] ?? currentVersion;
      final String apkUrl = data['apk_url'] ?? '';

      // Récupération de la clé API Gemini stockée sur Firestore
      geminiApiKey = data['gemini_api_key'] ?? '';

      // Comparaison logique des versions segment par segment
      if (_isVersionNewer(currentVersion, latestVersion) && apkUrl.isNotEmpty) {
        if (context.mounted) {
          _showUpdateDialog(context, latestVersion, apkUrl);
        }
      }
    } catch (e) {
      debugPrint("Erreur lors de la vérification de version : \$e");
    }
  }

  static bool _isVersionNewer(String current, String latest) {
    try {
      List<int> currentParts = current.split('.').map(int.parse).toList();
```

```
        List<int> latestParts = latest.split('.').map(int.parse).toList();

        for (int i = 0; i < 3; i++) {
          int currentPart = i < currentParts.length ? currentParts[i] : 0;
          int latestPart = i < latestParts.length ? latestParts[i] : 0;

          if (latestPart > currentPart) return true;
          if (latestPart < currentPart) return false;
        }
      } catch (e) {
        debugPrint("Erreur de parsing de version : \$e");
      }
      return false;
    }

    static void _showUpdateDialog(BuildContext context, String latestVersion, String apkUrl) {
      showDialog(
        context: context,
        barrierDismissible: false, // Empêche la fermeture hors du cadre
        builder: (BuildContext context) {
          return PopScope(
            canPop: false, // Bloque le bouton retour physique Android
            child: AlertDialog(
              title: const Text("Mise à jour obligatoire"),
              content: Text("Une nouvelle version (\$latestVersion) est disponible. Veuillez la t
              actions: [
                ElevatedButton(
                  onPressed: () async {
                    final uri = Uri.parse(apkUrl);
                    if (await canLaunchUrl(uri)) {
                      await launchUrl(uri, mode: LaunchMode.externalApplication);
                    }
                  },
                  child: const Text("TÉLÉCHARGER LA MISE À JOUR"),
                ),
              ],
            ),
          );
        },
      );
    }
  }
```

4.3 Le moteur d'intelligence artificielle : chat_engine.dart

Le moteur de chat asynchrone gère l'appel à l'API de Gemini 2.5 Flash. Il intègre de manière transparente la clé API récupérée dynamiquement de Firestore et convertit les images jointes en

Base64 pour l'analyse multimodale directe de l'IA.

```
import 'dart:convert';
import 'dart:io';
import 'package:http/http.dart' as http;
import 'package:flutter/foundation.dart';
import 'package:cloud_firestore/cloud_firestore.dart';
import 'package:securia/core/version_helper.dart';

class ChatEngine {
  static const String _apiBaseUrl =
      "https://generativelanguage.googleapis.com/v1beta/models/gemini-2.5-flash:generateContent

  Future<String> sendMessage(
    String userPrompt, {
    List<Map<String, dynamic>> history = const [],
    String? attachmentPath,
    String? attachmentType,
  }) async {
    // 1. Récupération de la clé API Gemini
    String apiKey = VersionHelper.geminiApiKey;
    if (apiKey.isEmpty) {
      // Chargement de secours direct depuis Firestore
      final doc = await FirebaseFirestore.instance.collection('app_config').doc('version_contro
      apiKey = doc.data()?['gemini_api_key'] ?? '';
      VersionHelper.geminiApiKey = apiKey;
    }

    if (apiKey.isEmpty) {
      return "Clé d'API Gemini introuvable. Veuillez configurer le champ 'gemini_api_key' dans
    }

    final String apiUrl = "\$_apiBaseUrl?key=\$apiKey";
    final List<Map<String, dynamic>> currentParts = [{"text": userPrompt}];

    // 2. Gestion de l'attachement d'image (Multimodalité Base64)
    if (attachmentPath != null && attachmentType == 'image') {
      final bytes = await File(attachmentPath).readAsBytes();
      final base64Image = base64Encode(bytes);
      currentParts.add({
        "inlineData": {
          "mimeType": attachmentPath.toLowerCase().endsWith(".png") ? "image/png" : "image/jpeg
          "data": base64Image
        }
      });
    }

    // 3. Payload d'appel API
    final payload = {
      "systemInstruction": {
        "parts": [{"text": "Tu es Pocam237, assistant IA de la police..."}]
      },
      "contents": [
```

```
          // Historique de session injecté ici...
          {"role": "user", "parts": currentParts}
        ],
        "generationConfig": {
          "temperature": 0.3,
          "maxOutputTokens": 8192
        }
      };

      final response = await http.post(
        Uri.parse(apiUrl),
        headers: {'Content-Type': 'application/json'},
        body: json.encode(payload),
      );

      if (response.statusCode == 200) {
        final result = json.decode(response.body);
        return result['candidates'][0]['content']['parts'][0]['text'];
      }
      return "Erreur HTTP \${response.statusCode} lors de l'appel de l'IA.";
    }
  }
```

Chapitre 5 : Intégration et Sécurisation de l'Écosystème Firebase

5.1 Création et liaison du projet Firebase

L'écosystème de SECUR.IA utilise Firebase comme solution backend sans serveur (Serverless). Pour lier votre application Flutter à un nouveau projet Firebase, la méthode officielle et automatique consiste à utiliser le **FlutterFire CLI**.

Étape 1 : Connexion au compte Firebase

Installez les outils Firebase (Firebase CLI) via Node.js sur votre machine de développement, puis connectez-vous avec votre compte de projet :

```
npm install -g firebase-tools
firebase login --reauth
```

Étape 2 : Activation globale de FlutterFire CLI

Activez la ligne de commande FlutterFire pour automatiser les liaisons dans le projet Dart :

```
dart pub global activate flutterfire_cli
```

Si la commande `flutterfire` n'est pas reconnue en raison de vos variables d'environnement PATH sous Windows, vous pouvez contourner cela en exécutant directement la commande via Dart :

```
dart pub global run flutterfire_cli:flutterfire configure
```

L'assistant interactif de configuration va alors analyser vos projets Firebase en ligne. Choisissez votre projet, sélectionnez les plateformes requises (**Android** et **iOS**) et validez. L'outil va générer

automatiquement les fichiers indispensables :

- `lib/firebase_options.dart` contenant les configurations d'initialisation de l'application.
- `android/app/google-services.json` pour le système natif Android.
- `ios/Runner/GoogleService-Info.plist` pour iOS.

5.2 Configuration des services sur la Console Firebase

A. Firebase Authentication (Accès Utilisateurs)

Pour sécuriser l'accès de l'application à vos seuls collaborateurs :

1. Rendez-vous sur votre **Console Firebase > Build > Authentication**.
2. Allez sur l'onglet **Sign-in method**.
3. Cliquez sur **Adresse e-mail/Mot de passe** (Email/Password), cochez **Activer** et enregistrez.

Vous pouvez maintenant inscrire manuellement les e-mails professionnels de vos agents de terrain depuis l'onglet **Users** pour leur donner l'accès à SECUR.IA.

B. Cloud Firestore (Base de données)

Firestore sert à stocker et diffuser les versions autorisées et la clé de l'IA.

1. Sur votre console Firebase, allez dans **Build > Firestore Database** et cliquez sur **Créer une base de données**.
2. Choisissez votre zone géographique de données et activez la base.
3. Créez la collection `app_config`, ajoutez-y le document `version_control` contenant les champs `current_version` (String), `apk_url` (String) et `gemini_api_key` (String).

> **Attention au vol de Clé API !**
>
> Ne commitez jamais de fichier contenant votre clé API Gemini sur des dépôts publics comme GitHub ou GitLab. Les robots de sécurité de Google les détectent en quelques minutes et révoquent automatiquement la clé. L'hébergement de la clé sur Firestore élimine ce risque à 100%.

5.3 Sécurisation : Les Règles de sécurité Firestore

(Rules)

Par défaut, Firestore bloque toutes les lectures et écritures. Vous devez définir des règles précises pour autoriser uniquement l'application mobile à consulter les versions et la clé de l'IA (en lecture seule), tout en interdisant à quiconque de modifier ces paramètres importants en dehors de votre console Firebase d'administration.

Allez dans l'onglet **Règles** (Rules) de votre Firestore sur la console Firebase et publiez le code suivant :

```
rules_version = '2';
service cloud.firestore {
  match /databases/{database}/documents {

    // Règle d'accès publique pour le contrôle de version et la clé API
    // Seul le document version_control est lisible par tous au démarrage de l'app.
    match /app_config/version_control {
      allow read: if true;
      allow write: if false; // Toute écriture extérieure est interdite (Lecture seule)
    }

    // Règle d'accès sécurisé pour les profils ou données d'utilisateurs
    // Nécessite que l'utilisateur soit authentifié via Firebase Auth
    match /{document=**} {
      allow read, write: if request.auth != null;
    }
  }
}
```

Chapitre 6 : Modèle Économique, Commercialisation et Impact

6.1 Le Modèle de commercialisation B2G (Business-to-Government)

Le marché de la technologie de sécurité publique obéit à des logiques contractuelles et politiques spécifiques. SECUR.IA n'est pas une application grand public distribuée de manière classique sur les stores applicatifs (Play Store ou App Store) ; sa distribution s'organise selon un modèle d'affaires de type **B2G (Business-to-Government)** et **B2B (Business-to-Business)** pour les flottes de sécurité privées.

La commercialisation de SECUR.IA s'articule autour de trois offres principales :

1. **Licence SaaS Souveraine (Software as a Service)** : Une redevance annuelle par agent déployé. Cette formule inclut la mise à jour en temps réel de la base de données d'annuaire, l'accès permanent aux modèles d'IA, et la maintenance corrective et évolutive de l'application.
2. **Déploiement On-Premise / Cloud Souverain (Clé en main)** : Pour les États ou administrations exigeant un cloisonnement absolu de leurs données (sécurité nationale). Le système Firebase/Firestore et les modèles d'IA de type open-source (comme Llama 3) sont déployés et configurés localement sur les serveurs physiques ou le cloud souverain de l'État acquéreur.
3. **Prestation de personnalisation (Setup Fee)** : Frais d'initialisation pour modéliser la structure de commandement, injecter l'annuaire existant, personnaliser la charte graphique et concevoir le prompt système de l'assistant IA selon les textes de lois spécifiques du pays.

6.2 Structure de Coûts et de Tarification

Pour garantir une rentabilité solide tout en proposant des tarifs attractifs pour les administrations publiques africaines, SECUR.IA adopte un modèle de tarification dégressif basé sur la taille de la flotte d'agents :

Taille de la Flotte (Agents actifs)	Frais de Personnalisation Initiale (Setup)	Redevance Annuelle (par agent / an)	Support & Maintenance (incluse)
Moins de 1 000 agents (Mairies, Communes)	5 000 000 FCFA (approx. 7 600 €)	3 500 FCFA (approx. 5,30 €)	Support standard par e-mail et WhatsApp
1 000 à 10 000 agents (Grandes Villes, Police Municipale)	12 000 000 FCFA (approx. 18 300 €)	2 500 FCFA (approx. 3,80 €)	Support dédié 24h/24 et 7j/7
Plus de 10 000 agents (Police Nationale, Gendarmerie)	Sur devis d'intégration sur-mesure	1 500 FCFA (approx. 2,30 €)	Ingénieur support sur site et SLA garanti

> *"La transformation numérique des services de sécurité publique en Afrique est un investissement hautement rentable. Elle réduit les temps de réponse de 40% et élimine les inefficacités opérationnelles liées à la lenteur d'accès aux procédures."*
>
> **Cabinet de Conseil en Innovation Digitale, Dakar**

6.3 Comment adapter SECUR.IA à son propre pays en Afrique ?

L'un des plus grands atouts techniques de SECUR.IA est sa modularité extrême. Pour déployer la solution dans un autre pays africain (comme la Côte d'Ivoire, le Sénégal ou le Gabon), le travail se résume en trois adaptations simples sans réécriture complète de l'application Flutter :

- **Étape 1 : Mettre à jour l'annuaire local JSON** : Remplacez les données de contact du fichier [annuaire.json](file:///d:/Projets%20Informatiques/rescue/securia/assets/data/annuaire.json) par les départements, communes, villes et numéros professionnels du pays cible.
- **Étape 2 : Adapter le prompt système de l'IA** : Dans le fichier [chat_engine.dart](file:///d:/Projets%20Informatiques/rescue/securia/lib/screens/ia/chat_engine.dart), modifiez

la constante `_systemPrompt` pour renommer l'assistant (par ex. *PoliSenegal* ou *GendIvoire*) et lui ordonner de répondre selon le Code Pénal du pays en question.

- **Étape 3 : Ajuster l'identité visuelle** : Modifiez la palette de couleurs dans [app_colors.dart](file:///d:/Projets%20Informatiques/rescue/securia/lib/core/app_colors.dart) pour adopter les couleurs nationales ou institutionnelles de l'administration cliente, et changez les logos dans le dossier d'images d'assets.

Chapitre 7 : Feuille de Route et Perspectives d'Évolution

7.1 Feuille de route technologique à 3 ans

Le développement de SECUR.IA est pensé comme une démarche d'innovation incrémentale. L'application actuelle pose des bases solides (accès hors-ligne, chatbot multimodal, configuration dynamique). La feuille de route technologique pour les trois prochaines années s'organise autour de quatre axes majeurs :

Phase A : Géolocalisation et Cartographie Tactique (Horizon 12 mois)

L'intégration de la **géolocalisation** permettra de localiser en temps réel les patrouilles sur le terrain et d'afficher les incidents à proximité sur une carte interactive locale (chargée en cache). Les agents pourront également utiliser un bouton de détresse ("SOS Opérationnel") pour partager instantanément leurs coordonnées géographiques avec les unités de renfort les plus proches.

Phase B : Notifications Push et Alertes de Masse (Horizon 18 mois)

En intégrant **Firebase Cloud Messaging (FCM)**, les directions régionales et nationales de police pourront envoyer des bulletins de sécurité en temps réel, des alertes de recherche (personnes recherchées, véhicules volés) ou des messages urgents de mobilisation générale directement dans l'application, y compris en arrière-plan.

Phase C : Chiffrement de bout en bout et Souveraineté Totale (Horizon 24 mois)

Mise en place de protocoles de chiffrement de bout en bout (type Signal Protocol) pour toutes les communications de l'annuaire interne et les fils de discussion d'IA, garantissant qu'aucune entité extérieure — pas même l'hébergeur cloud — ne puisse intercepter les données stratégiques échangées par les agents.

Phase D : Raccordement aux bases nationales (Horizon 36 mois)

Connexion sécurisée (via des API VPN d'État) de l'assistant IA avec les fichiers nationaux autorisés (cartes grises, fichiers d'immatriculation, fichiers de recherche judiciaires) pour permettre aux OPJ de mener des vérifications de conformité administratives de manière instantanée par simple saisie vocale ou capture d'image sur le terrain.

7.2 Conclusion : L'Afrique au cœur de l'innovation numérique souveraine

SECUR.IA démontre que l'intelligence artificielle n'est plus une technologie lointaine réservée aux centres de recherche des pays les plus riches. Grâce à la flexibilité de Flutter, à la puissance de calcul abordable de l'API de Gemini et à l'intelligence de conception asymétrique hors-ligne, il est possible de concevoir et de déployer en quelques mois des outils de sécurité de niveau mondial répondant aux réalités concrètes du continent africain.

En mettant l'innovation numérique au service de ceux qui protègent la nation, SECUR.IA pave la voie à une nouvelle ère de gouvernance publique technologique : plus réactive, plus efficace, plus juste, et résolument souveraine.

> *"Le futur de la sécurité publique en Afrique s'écrira en lignes de code souveraines. SECUR.IA est le premier pas vers cette émancipation technologique."*
>
> **Directeur Général de la Sécurité Nationale, Vision 2030**